Received: 6 April 2021 | Revised: 12 August 2021 | Accepted: 25 October 2021

DOI: 10.1002/we.2694

RESEARCH ARTICLE

WILEY

Streaming dynamic mode decomposition for short-term forecasting in wind farms

Jaime Liew | Tuhfe Göçmen | Wai Hou Lio | Gunner Chr. Larsen

Department of Wind Energy, Technical University of Denmark, Roskilde, Denmark

Correspondence
Jaime Liew, Department of Wind Energy, Technical University of Denmark, Frederiksborgvej 399, 4000 Roskilde, Denmark.
Email: jyli@dtu.dk

Funding information
WISDOM Project, Grant/Award Number: 12842

Abstract

Forecasting in wind energy is a crucial task to perform adequate wind farm flow control or to participate in the energy market. While many power forecasting methods exist, it is notoriously difficult to capture both short- and long-term variations in the wind farm system in real time. We demonstrate a data-driven real-time system identification approach to forecasting based on streaming dynamic mode decomposition methodology (sDMD). The method is capable of characterizing nonlinear, time-varying, multidimensional time series data in a computationally efficient manner. The algorithm is modified to work with data streams by adjusting the dynamic mode decomposition continuously as new data are made available. The method is applied to high-frequency SCADA data from the Lillgrund offshore wind farm. A 23.31% improvement over persistence forecasting is found for 5-min-ahead forecasts of the power output of all turbines in the wind farm. sDMD is shown to be a suitable tool for capturing short-term dynamics while adapting to long-term changes in wind speed and direction and has potential applications in real-time wind farm control.

KEYWORDS
dynamic mode decomposition, forecasting, real time, wind energy

1 | INTRODUCTION

Wind energy generation has shown impressive growth over the past decade due to rapidly decreasing costs and steadily increasing global demand for renewable energy solutions.[1] To meet the growing demand for efficient wind energy generation, accurate forecasts are crucial. Wind power forecasting is still a major research and implementation challenge, as it is essential for both the planning and operation stages of a wind power plant. In particular, short-term forecasts are typically utilized for active turbine and wind farm control[2] and power trading in the electricity markets.[3] Due to shorter timescales, an important aspect for short-term wind power forecasting is the energy losses due to aerodynamic interactions between the wind turbines in a wind power plant. These wake losses can cause reductions in single turbine power output by up to 70% depending on the wind direction and turbine spacing.[4] Especially for control applications, where the future state of the system is to be predicted, the nonlinear temporal and spatial dynamic behavior of the wake remains a challenge to model. The difficulty originates largely from turbulence, wake effects, and changing wind conditions.

While wake effects present a challenge in wind farm planning, they also propagate information through the wind farm, which can be used as a means for power forecasting at the turbine or plant level. It is expected that information (wind speed, wind direction, and power output) from upstream turbines can be leveraged to infer the future state of downstream turbines. Wind farm power forecasting within a horizon of seconds to minutes can benefit from this information propagation; however, a number of challenges arise. Firstly, a model which is able to accurately

This is an open access article under the terms of the Creative Commons Attribution-NonCommercial License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited and is not used for commercial purposes.
© 2021 The Authors. *Wind Energy* published by John Wiley & Sons Ltd.

characterize the fluid–structure interactions between turbines requires a high-dimensional state and immense computing power which is intractable for real-time purposes. Secondly, the wind farm system is time-varying due to changes in wind speed, wind direction, and the spectral content of the turbulence. Forecasters must therefore also be time-varying and able to adapt to changing conditions.

At the core of a wind farm forecaster is a wind farm model. The literature on wind farm modeling, both physics-based and data-driven, is vast and ever-growing. A typical physics-based wind farm model uses a combination of rotor models[5] and wake models.[6-9] It is notoriously difficult to match a physics-based model to reality without extensive calibration. For this reason, purely data-driven approaches are increasingly available in the wind energy field. In addition to the more conventional statistical approaches such as autoregression,[10,11] artificial intelligence and machine learning applications are growing in popularity. The general sensitivity in model-free methods lies within the availability and quality of the training data. For the machine learning applications, not only in wind but generally in renewable energy forecasting, the comprehensive review by Voyant et al[12] highlights the promising results obtained by support vector machines,[13,14] regression trees,[15] and random forests.[16,17] The most recent review concerning the deep learning applications in renewable energy forecasting[18] compares several architectures implemented in a number of case studies and shows that the deep recurrent neural networks outperform the others both on root-mean-square error (RMSE) and mean absolute percentage error indices. A recent application of recurrent neural networks with transfer learning capabilities for turbine power forecasting is presented in Göçmen et al.[19] However, the applied algorithms are increasingly complex, reducing the interpretability and explainability of the intelligent forecasting approaches and increasing the risk.

At the intersection of model-based and model-free methods exists several data-informed modeling approaches for flow predictions and forecasting that brings together the physical models and observations. These hybrid approaches typically include model adaptations via active learning from operational data to correct potential physical model inadequacies. For example, Ben Bouallègue et al[20] apply a dual-ensemble copula-coupling approach to correct the numerical weather prediction (NWP) ensembles using the autocorrelation error estimated from historical data. Eide et al[21] employ Bayesian model updates to the physics-based ensemble NWP for better probabilistic wind speed estimations. Andersson and Imsland[22] adopt a similar approach using Gaussian process regression with modifier adaptation for short-term flow prediction and control. However, active model adaptations have a limited effect in improving the accuracy of the predictions if the adjustable parameters in the physical model are structurally insufficient to accommodate such updates.[23,24]

Regardless of the type of model used, a well-function forecaster for wind energy applications must be able to adapt to changing system conditions. Long-term changes to the wind speed, wind direction, turbulence, atmospheric stability, and turbine depreciation can degrade the performance of a wind farm model. Adaptive filters, which adjust their transfer function parameters in real time, play an important role in this application. Among the adaptive filters, the recursive least squares (RLS) filter is well regarded for its ability to solve a least squares problem incrementally, while being computational efficient.[25,26] Unfortunately, RLS is vulnerable to ill-conditioning due to rank-deficient observations, which is often the case in wind energy applications when the provided signals originate from a large sample space or there exists data redundancy.[27] The method proposed in this investigation shares many qualities with the RLS filter and overcomes some of the shortcomings it faces by adopting the methodologies of dynamic mode decomposition.[28,29]

Dynamic mode decomposition (DMD) is an increasingly popular method for characterizing spatiotemporal behavior in a system given snapshots of the system state. The advantage of using a DMD approach is its ability to identify low-rank behavior in a high-dimensional system, which greatly reduces the need for expensive computations. The behavior of a system is characterized as the superposition of DMD modes. Each mode has a spatial behavior, which evolves in time at a single frequency with an exponentially growing or decaying amplitude. Furthermore, a DMD analysis of a system is compatible with classical control theory as it is able to estimate the system behavior as a linear dynamic system. DMD, which initially showed success in the field of fluid dynamics, has been applied to a wide range of disciplines including neuroscience, economics, and epidemiology. In the field of wind energy, DMD is also showing increasing popularity.[30-32] Annoni et al. used the DMD methodology to perform sparse optimization of the location of wind sensors to reconstruct a wind farm wind field.[30] Cassamo and van Wingerden show the potential benefits of using DMD for wake reconstruction in dynamic induction control.[32] Whereas these studies estimate the wind field, the present study takes a different approach by reconstructing the time series of interest directly from the high-frequency SCADA data of each individual turbine in a wind farm environment. In particular, we use a combination of the wind power output, wind speed, and wind direction of each turbine to forecast the power output of each turbine.

In this study, a variant of the standard DMD algorithm is used which is able to accommodate streaming data. Streaming DMD (sDMD, also known as online DMD[33] or on-the-fly DMD[34]) continuously updates the DMD system matrices as new data become available. This is in contrast to the "batch-processed" methodology, which is typically used in machine learning applications. While non-streaming versions of DMD are by far the most popular in literature, there are several methods adapted for streaming data. Zhang et al. propose a computationally efficient method for updating the exact DMD modes using a rank-1 update as each new observation is obtained.[33] This method, however, does not include the model reduction step in DMD, which is typically performed using a singular value decomposition (SVD). Thus, the algorithm operates on high-dimensional system states and shares similarities with RLS. In contrast, Matsumoto and Indinger adopt a streaming SVD, but it does not include a streaming implementation of the regression step of DMD.[34] A third variation, which contains both a streaming SVD and a streaming regression step, is proposed by Hemati et al.[35] In this study, we adopt the method proposed by Hemati et al. while including an exponential memory method proposed by Zhang et al.

Two additional variations of DMD are briefly explored. The first is the use of delay states, which is achieved by arranging the snapshot states into Hankel blocks. The use of Hankel blocks in DMD allows for additional hidden oscillatory modes to be identified by increasing the order of the system.[36-38] In particular, Arbabi and Mezic show that DMD using Hankel matrix blocks can yield the true Koopman eigenfunctions and eigenvalues.[38] The second variation is the use of additional input signals. Whereas standard DMD is able to estimate the state matrix, **A**, in a state-space system, Proctor et al. demonstrate how to extend DMD to be able to estimate the input matrix, **B**.[39]

The presented study is arranged as follows. First, the algorithm for standard DMD and sDMD is described in Section 2. Next, a case study is presented where sDMD is applied to measured time series data of the Lillgrund offshore wind farm and evaluated in Section 3. The study is concluded with a discussion and conclusion of the results.

2 | METHOD

The presented sDMD algorithm is designed to perform a forecast f time steps into the future, which is typically in the timescale of minutes for the present application. The algorithm initializes with a standard DMD and continues indefinitely using the sDMD scheme. In this section, the standard DMD algorithm is first formulated, followed by an alternative, but equivalent reformulation. Finally, using the reformulation of DMD, a detailed description of the streaming DMD algorithm is provided.

2.1 | Standard formulation of dynamic mode decomposition

The DMD algorithm is able to characterize the spatiotemporal behavior of a system given a series of state snapshots. The presented interpretation of DMD is based on the definitions described in Tu et al.[37] Suppose we have two sets of data snapshots up to time step k,

$$\mathbf{X}_k = \begin{bmatrix} | & & | & | \\ \mathbf{x}_{k-w+1} & \dots & \mathbf{x}_{k-1} & \mathbf{x}_k \\ | & & | & | \end{bmatrix} \qquad \mathbf{Y}_k = \begin{bmatrix} | & & | & | \\ \mathbf{x}_{k+f-w+1} & \dots & \mathbf{x}_{k+f-1} & \mathbf{x}_{k+f} \\ | & & | & | \end{bmatrix} \tag{1}$$

where, $\mathbf{X}_k, \mathbf{Y}_k \in \mathbb{R}^{n\times w}$, n is the number of states, and w is the number of observations. The columns of $\mathbf{X}_k$ to $\mathbf{Y}_k$ consist of state snapshots which are offset by f time steps. A best fit linear operator mapping the observations from $\mathbf{X}_k$ to $\mathbf{Y}_k$ is denoted as $\mathbf{A}_k$, such that

$$\mathbf{Y}_k \approx \mathbf{A}_k \mathbf{X}_k \tag{2}$$

where $\tilde{\mathbf{A}}_k$ is chosen to minimizing the cost function

$$\|\mathbf{Y}_k - \mathbf{A}_k \mathbf{X}_k\|_F \tag{3}$$

where $\|\cdot\|_F$ is the Frobenius norm. While it is possible to perform the minimization as

$$\mathbf{A}_k = \mathbf{Y}_k \mathbf{X}_k^{\dagger} \tag{4}$$

where $\dagger$ denotes the Moore–Penrose pseudoinverse,[40] it is computationally more tractable to approximate $\mathbf{A}_k$ as follows: First, take the truncated singular value decomposition (SVD) of $\mathbf{X}_k$ up a rank, r:

$$\mathbf{X}_k = [\mathbf{U}_k \;\; \mathbf{U}_{rem}] \begin{bmatrix} \boldsymbol{\Sigma}_k & 0 \\ 0 & \boldsymbol{\Sigma}_{rem} \end{bmatrix} \begin{bmatrix} \mathbf{V}_k^{\mathsf{T}} \\ \mathbf{V}_{rem}^{\mathsf{T}} \end{bmatrix} \tag{5}$$

where $\mathbf{U}_k \in \mathbb{R}^{n\times r}, \boldsymbol{\Sigma}_k \in \mathbb{R}^{r\times r}, \mathbf{V}_k \in \mathbb{R}^{w\times r}, \mathbf{U}_{rem} \in \mathbb{R}^{n\times(n-r)}, \boldsymbol{\Sigma}_{rem} \in \mathbb{R}^{(n-r)\times(w-r)}$, and $\mathbf{V}_{rem} \in \mathbb{R}^{w\times(w-r)}$. $\mathbf{X}_k$ can therefore be approximated as

$$\mathbf{X}_k \approx \mathbf{U}_k \boldsymbol{\Sigma}_k \mathbf{V}_k^{\mathsf{T}} \tag{6}$$

and Equation (4) can therefore be approximated as

$$\mathbf{A}_k \approx \mathbf{Y}_k \mathbf{V}_k \boldsymbol{\Sigma}_k^{-1} \mathbf{U}_k^{\mathsf{T}} \tag{7}$$

however, in practice, it is computationally more convenient to compute a projection of the transition matrix,

$$\tilde{\mathbf{A}}_k \approx \mathbf{U}_k^{\mathsf{T}} \mathbf{Y}_k \mathbf{V}_k \boldsymbol{\Sigma}_k^{-1} \tag{8}$$

The DMD modes can then be computed in the reduced order space by taking the eigenvalue decomposition of $\tilde{\mathbf{A}}_k$, which is well described in DMD literature.[41]

2.2 | Alternative formulation of dynamic mode decomposition

In this section, an alternate, but equivalent formulation of DMD is presented to accommodate the sDMD algorithm in the next section. Consider the same data snapshots in Equation (1). First, we perform the truncated SVD of both $\mathbf{X}_k$ and $\mathbf{Y}_k$ separately, to truncation ranks of r_X and r_Y, respectively:

$$\mathbf{X}_k \approx \mathbf{U}_k^X \boldsymbol{\Sigma}_k^X \mathbf{V}_k^{X\mathsf{T}} \qquad \mathbf{Y}_k \approx \mathbf{U}_k^Y \boldsymbol{\Sigma}_k^Y \mathbf{V}_k^{Y\mathsf{T}} \tag{9}$$

Then, the reduced-order snapshot matrices are computed by projecting the high-dimensional data onto the column space of $\mathbf{U}_k^X$ and $\mathbf{U}_k^Y$:

$$\tilde{\mathbf{X}}_k = \mathbf{U}_k^{X\mathsf{T}} \mathbf{X}_k \qquad \tilde{\mathbf{Y}}_k = \mathbf{U}_k^{Y\mathsf{T}} \mathbf{Y}_k \tag{10}$$

The reduced-order transition matrix, $\tilde{\mathbf{A}}_k \in \mathbb{R}^{r \times r}$, can then be computed using the pseudoinverse operation:

$$\tilde{\mathbf{A}}_k = \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^{\dagger} \tag{11}$$

$\tilde{\mathbf{A}}_k$ in Equation (11) can be shown to be exactly equivalent to that in Equation (8) as follows. First, expand the pseudoinverse operation in Equation (11) as

$$\tilde{\mathbf{A}}_k = \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}} \left(\tilde{\mathbf{X}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}} \right)^{-1} \tag{12}$$

We first note that, from Equations (9), (10), and (12), the matrix inversion component can be expressed as

$$\left(\tilde{\mathbf{X}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}} \right)^{-1} = \left(\mathbf{U}_k^{X\mathsf{T}} \mathbf{X}_k \mathbf{X}_k^{\mathsf{T}} \mathbf{U}_k^X \right)^{-1} \tag{13}$$

$$= \left(\mathbf{U}_k^{X\mathsf{T}} \mathbf{U}_k^X \boldsymbol{\Sigma}_k^X \mathbf{V}_k^{X\mathsf{T}} \mathbf{V}_k^X \boldsymbol{\Sigma}_k^X \mathbf{U}_k^{X\mathsf{T}} \mathbf{U}_k^X \right)^{-1} \tag{14}$$

$$= \left(\boldsymbol{\Sigma}_k^X \right)^{-2} \tag{15}$$

noting that $\mathbf{U}_k^{X\mathsf{T}} \mathbf{U}_k^X = \mathbf{V}_k^{X\mathsf{T}} \mathbf{V}_k^X = \mathbf{I}$ as they are semi-orthogonal matrices.[42] Substituting Equation (15) into Equation (12) yields the following simplification:

$$\tilde{\mathbf{A}}_k = \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}} \left(\tilde{\mathbf{X}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}} \right)^{-1} \tag{16}$$

$$= \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}} \left(\boldsymbol{\Sigma}_k^X \right)^{-2} \tag{17}$$

$$= \mathbf{U}_k^{Y\mathsf{T}} \mathbf{Y}_k \mathbf{V}_k \boldsymbol{\Sigma}_k^X \mathbf{U}_k^{X\mathsf{T}} \mathbf{U}_k^X \left(\boldsymbol{\Sigma}_k^X \right)^{-2} \tag{18}$$

$$= \mathbf{U}_k^{Y\mathsf{T}} \mathbf{Y}_k \mathbf{V}_k \left(\mathbf{\Sigma}_k^X\right)^{-1} \tag{19}$$

When $r_X = r_Y$, and the snapshots from $\mathbf{X}_k$ and $\mathbf{Y}_k$ differ by a single column, which is typically the case for DMD applications, $\mathbf{U}_k^X$ and $\mathbf{U}_k^Y$ are approximately equal, and therefore, Equations (19) and (8) are equivalent. This alternative formulation DMD is convenient as it can be performed without the need of $\mathbf{V}_k$, which can reduce the required memory resources when there are large quantities of snapshots.

It should be noted that having separate decompositions for $\mathbf{X}_k$ and $\mathbf{Y}_k$ in Equation (10) allows the input and output subspaces to have different sizes and span different bases. This generalization, matching the work of Proctor et al[39] and Hemati et al,[35] is useful when there are more input channels than output channels. Additionally, for forecasts many time steps ahead, there may be a significant delay between the input and output signals. This extension of DMD accommodates the definition of streaming DMD in the following section.

2.3 | Streaming dynamic mode decomposition

sDMD is a modification of the standard DMD algorithm to accommodate for streaming data. Whereas standard DMD can be performed on a chunk of data, sDMD can update its decomposition successively as new information is obtained. The resultant method is a computationally efficient adaptive method for updating the DMD mode indefinitely. Equation (11) is equivalently represented as the product of two matrices, $\mathbf{Q}_k \in \mathbb{R}^{r_Y \times r_X}$, $\mathbf{P}_k^X \in \mathbb{R}^{r_X \times r_X}$.

$$\tilde{\mathbf{A}}_k = \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}} \left(\tilde{\mathbf{X}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}}\right)^{-1} \tag{20}$$

$$\tilde{\mathbf{A}}_k = \mathbf{Q}_k \left(\mathbf{P}_k^X\right)^{-1} \tag{21}$$

where $\mathbf{Q}_k = \tilde{\mathbf{Y}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}}$, and $\mathbf{P}_k^X = \tilde{\mathbf{X}}_k \tilde{\mathbf{X}}_k^{\mathsf{T}}$. A requirement for Equation (20) to be valid is that $\tilde{\mathbf{X}}_k$ must be full rank. The SVD truncation rank, r_X, must therefore be small enough to achieve this condition. $\mathbf{Q}_k$ and $\mathbf{P}_k^X$ are stored in memory and updated continually as new snapshot pairs are measured. An additional matrix, $\mathbf{P}_k^Y \in \mathbb{R}^{r_Y \times r_Y}$, is also calculated at initialization as $\mathbf{P}_k^Y = \tilde{\mathbf{Y}}_k \tilde{\mathbf{Y}}_k^{\mathsf{T}}$. This matrix is required for the rank reduction step described below. When a new pair of observations, $\mathbf{x}_{k+1}$ and $\mathbf{y}_{k+1}$ are obtained, $\mathbf{Q}_{k+1}$, $\mathbf{P}_{k+1}^X$, and $\mathbf{P}_{k+1}^Y$ are updated by performing rank-1 updates as follows:

$$\mathbf{Q}_{k+1} = \rho \mathbf{Q}_k + \mathbf{y}_{k+1} \mathbf{x}_{k+1}^{\mathsf{T}} \tag{22}$$

$$\mathbf{P}_{k+1}^X = \rho \mathbf{P}_k^X + \mathbf{x}_{k+1} \mathbf{x}_{k+1}^{\mathsf{T}} \tag{23}$$

$$\mathbf{P}_{k+1}^Y = \rho \mathbf{P}_k^Y + \mathbf{y}_{k+1} \mathbf{y}_{k+1}^{\mathsf{T}} \tag{24}$$

where $\rho \in [0, 1]$ is the "forgetting factor." The updating scheme in Equations (22)–(24) is described in Zhang et al.[33] If the model reduction step is omitted, the method is equivalent to multidimensional recursive least squares. The inclusion of ρ causes the DMD to consider the most recent observations with a higher weight. It modifies the regression cost function from Equation (3) as follows:

$$\sum_{i=k}^{k+w-1} \rho^{k+w-1-i} \left\| \tilde{\mathbf{y}}_i - \tilde{\mathbf{A}}_k \tilde{\mathbf{x}}_i \right\|_2^2 \tag{25}$$

In this study, ρ is chosen based on a memory half-life parameter, $T_{1/2}$, where $\rho = 2^{-T_s/T_{1/2}}$, which is the time elapsed before the weight of a sample is halved. T_s is the sampling time of the algorithm. When $\rho = 1$ ($T_{1/2} \to \infty$), all samples collected since initialization are weighted equally. Over a large number of updates, this can lead to numerical instabilities due to the large range of magnitudes in Equation (21). The inclusion of a memory half-life allows the algorithm to run indefinitely by constraining the magnitude of matrices $\mathbf{P}_k^X$, $\mathbf{P}_k^Y$, and $\mathbf{Q}_k$.

To keep the low-order projection up to date, the projection error is determined before the execution of Equations (22) and (23). The projection error quantifies the information loss from projecting the higher dimensional state to a low-dimensional space and is calculated as[35]

$$e_x = \frac{\|\mathbf{x} - \mathbf{U}_k^X \mathbf{U}_k^{X\mathrm{T}} \mathbf{x}\|_2}{\|\mathbf{x}\|_2} \tag{26}$$

If the projection error exceeds a user-defined threshold, ϵ, the rank of the basis, $\mathbf{U}_k^X$, is augmented by adding one basis vector to the column space. As $\mathbf{x} - \mathbf{U}_k^X \mathbf{U}_k^{X\mathrm{T}} \mathbf{x}$ is, by definition, orthogonal to the column space of $\mathbf{U}_k^X$, it is suitable to be used as the new basis vector. Additionally, $\mathbf{Q}_k$ and $\mathbf{P}_k^X$ must also be updated to take into account the new basis size. This is achieved by zero padding both matrices. Similarly, if the output basis projection error, $e_y = \|\mathbf{y} - \mathbf{U}_k^Y \mathbf{U}_k^{Y\mathrm{T}} \mathbf{y}\|_2 / \|\mathbf{y}\|_2$, exceeds the threshold, then the system matrices are updated in a similar way. The update schemes for the rank augmentations are described in Table 1.

After several rank augmentations, $\mathbf{U}_k^X$ and $\mathbf{U}_k^Y$ will approach full rank. This is undesirable, as the benefits of a low-order reduction are diminished. To avoid this, a second type of basis update is performed, where the basis size is reduced to r_{min}. This is achieved by reorthogonalizing the basis vectors using a proper orthogonal decomposition of the leading eigenvectors of the DMD system.[35] This can conveniently be achieved using an eigendecomposition $\mathbf{P}_k^X$ (or $\mathbf{P}_k^Y$). For instance, if $\mathbf{U}_k^X$ is to be rank reduced, eigendecompose $\mathbf{P}_k^X$ to obtain the leading $r_{\min}$ eigenvalues in a diagonal matrix, $\boldsymbol{\Lambda} \in \mathbb{R}^{r_{\min} \times r_{\min}}$, and the leading $r_{\min}$ eigenvectors as columns of $\boldsymbol{\psi}_k^X \in \mathbb{R}^{r_{\max} \times r_{\min}}$. The system matrices can be updated as described in Table 1.

A future state prediction, $\mathbf{y}'$, can be performed on an input state, $\mathbf{x}$, by performing:

$$\mathbf{y}' = \mathbf{U}_k^Y \tilde{\mathbf{A}}_k \mathbf{U}_k^{X\mathrm{T}} \mathbf{x} \tag{27}$$

where $\tilde{\mathbf{A}}_k$ is calculated using Equation (21). If the input and output signals share the same basis, we can observe that $\mathbf{U}_k^Y \tilde{\mathbf{A}}_k \mathbf{U}_k^{X\mathrm{T}}$ is the state transition matrix in the high-dimensional space. While it is possible to simplify Equation (27) by combining the first three matrices into one, it is computationally less expensive in terms of both memory and processing power to separate the operation into three matrix multiplications.

DMD modes can be retrieved if the input and output signals share the same basis. To retrieve the DMD modes of the system, determine the eigenvalues and eigenvectors of the reduced order transition matrix, $\mathbf{U}_k^{X\mathrm{T}} \mathbf{U}_k^Y \tilde{\mathbf{A}}$. If $\mathbf{v}_i$ is the ith eigenvector of $\mathbf{U}_k^{X\mathrm{T}} \mathbf{U}_k^Y \tilde{\mathbf{A}}$, then $\mathbf{U}_k^X \mathbf{v}_i$ is the ith DMD mode.

A Python implementation of the sDMD algorithm is available at https://doi.org/10.5281/zenodo.4646749.[43]

2.4 | State augmentation

In addition to the vector of values which are to be predicted, $\mathbf{x}_k$, there are often additional data available which can provide additional information, but do not to be predicted themselves. For example, for forecasting wind turbine power output, these additional channels could include

TABLE 1 SVD update scheme

	Update condition	Matrix update
Rank augmentation of $\mathbf{U}_k^X$	$\frac{\|\mathbf{x} - \mathbf{U}_k^X \mathbf{U}_k^{X\mathrm{T}} \mathbf{x}\|_2}{\|\mathbf{x}\|_2} > \epsilon$	$\mathbf{U}_k^X \leftarrow \begin{bmatrix} \mathbf{U}_k^X & \frac{\mathbf{x} - \mathbf{U}_k^X \mathbf{U}_k^{X\mathrm{T}} \mathbf{x}}{\|\mathbf{x} - \mathbf{U}_k^X \mathbf{U}_k^{X\mathrm{T}} \mathbf{x}\|_2} \end{bmatrix}$ $\mathbf{Q}_k \leftarrow \begin{bmatrix} \mathbf{Q}_k \\ \mathbf{0}^{1 \times r} \end{bmatrix}$ $\mathbf{P}_k^X \leftarrow \begin{bmatrix} \mathbf{P}_k^X & \mathbf{0}^{r \times 1} \\ \mathbf{0}^{1 \times r} & 0 \end{bmatrix}$
Rank augmentation of $\mathbf{U}_k^Y$	$\frac{\|\mathbf{y} - \mathbf{U}_k^Y \mathbf{U}_k^{Y\mathrm{T}} \mathbf{y}\|_2}{\|\mathbf{y}\|_2} > \epsilon$	$\mathbf{U}_k^Y \leftarrow \begin{bmatrix} \mathbf{U}_k^Y & \frac{\mathbf{y} - \mathbf{U}_k^Y \mathbf{U}_k^{Y\mathrm{T}} \mathbf{y}}{\|\mathbf{y} - \mathbf{U}_k^Y \mathbf{U}_k^{Y\mathrm{T}} \mathbf{y}\|_2} \end{bmatrix}$ $\mathbf{Q}_k \leftarrow \begin{bmatrix} \mathbf{Q}_k & \mathbf{0}^{r \times 1} \end{bmatrix}$ $\mathbf{P}_k^Y \leftarrow \begin{bmatrix} \mathbf{P}_k^Y & \mathbf{0}^{r \times 1} \\ \mathbf{0}^{1 \times r} & 0 \end{bmatrix}$
Rank reduction of $\mathbf{U}_k^X$	$\mathrm{rank}(\mathbf{U}_k^X) > r_{max}$	$\mathbf{U}_k^X \leftarrow \mathbf{U}_k^X \boldsymbol{\psi}_k^X$ $\mathbf{Q}_k \leftarrow \mathbf{Q}_k \boldsymbol{\psi}_k^X$ $\mathbf{P}_k^X \leftarrow \boldsymbol{\Lambda}_k^X$
Rank reduction of $\mathbf{U}_k^Y$	$\mathrm{rank}(\mathbf{U}_k^Y) > r_{max}$	$\mathbf{U}_k^Y \leftarrow \mathbf{U}_k^Y \boldsymbol{\psi}_k^Y$ $\mathbf{Q}_k \leftarrow \boldsymbol{\psi}_k^{Y\mathrm{T}} \mathbf{Q}_k$ $\mathbf{P}_k^Y \leftarrow \boldsymbol{\Lambda}_k^Y$

measurements of wind speed wind direction, turbine rotor speed, and blade pitch angles. Let $\mathbf{u}_k \in \mathbb{R}^{n_u}$ represent n_u additional channels. To incorporate $\mathbf{u}_k$ into the DMD forecast, the augmented input vector $\boldsymbol{\Omega}_k \in \mathbb{R}^{n+n_u}$ is used instead of the input vector described in Equation (9). $\boldsymbol{\Omega}_k$ is defined as

$$\boldsymbol{\Omega}_k = \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} \tag{28}$$

An additional state augmentation can be performed by including delay states. In the formulation outlined in the previous section, only the most recent state observation is used for the forecast. By adding additional past state observations, the sDMD forecasts can be strengthened. Delay states can be added by concatenating the previous $s+1$ states to the input vector. The input state matrix from Equation (9) is therefore replaced with

$$\mathbf{X}_k^{\text{AUG}} = \begin{bmatrix} \boldsymbol{\Omega}_{k-w+1} & \dots & \boldsymbol{\Omega}_{k-1} & \boldsymbol{\Omega}_k \\ \boldsymbol{\Omega}_{k-w} & \dots & \boldsymbol{\Omega}_{k-2} & \boldsymbol{\Omega}_{k-1} \\ \vdots & \vdots & \vdots & \vdots \\ \boldsymbol{\Omega}_{k-s-w+1} & \dots & \boldsymbol{\Omega}_{k-s-1} & \boldsymbol{\Omega}_{k-s} \end{bmatrix} \tag{29}$$

where $\mathbf{X}_k^{\text{AUG}} \in \mathbb{R}^{s(n+n_u)\times w}$.

2.5 | Method summary

The sDMD method described in this section is a modification to the standard DMD method by adjusting the DMD modes as new information is made available. Unlike the standard DMD method which requires a batch of data, sDMD is able to operate on a streaming data set. The presented formulation allows both the SVD and the linear regression steps to be updated in a computationally efficient manner. The user must choose the values of four parameters, ϵ, $T_{1/2}$, r_{min}, and r_{max}. The projection error threshold, ϵ, and the range of basis sizes, r_{min} and r_{max}, determine the frequency that the SVD basis is updated. r_{min} and r_{max} can be determined by performing an SVD on a subset of data and observing the required rank truncation to describe the data adequately. r_{max} must be low enough so that the reduced-order state is full rank; otherwise, the matrix inversion in Equation (20) is invalid.

The memory half-life, $T_{1/2}$, determines the rate at which new information is incorporated into the DMD modes. A longer half-life will give new observations a lower weight and vice versa. The user should choose $T_{1/2}$ based on the expected timescale of changes in the modeled system.

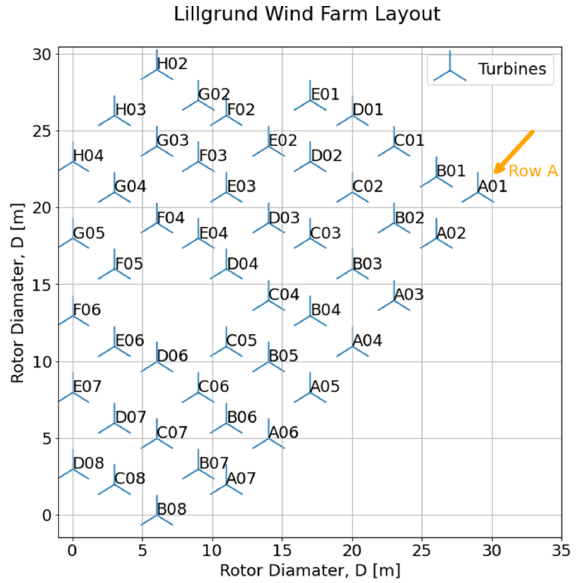
The presented formulation for sDMD is computationally efficient compared to non-streaming DMD and many machine learning algorithms such as neural networks. The computational complexity of batch DMD is $\mathcal{O}(n^3 + n^2 m)$. In contrast, sDMD has a complexity of approximately $\mathcal{O}(mnr)$ for all m time steps if the SVD update occurs infrequently. The model reduction step provides noticeable computational improvement over performing Exact DMD or RLS, which have a time complexity of $\mathcal{O}(mn^2)$, especially when n is large and $r \ll n$.

3 | CASE STUDY—LILLGRUND WIND FARM

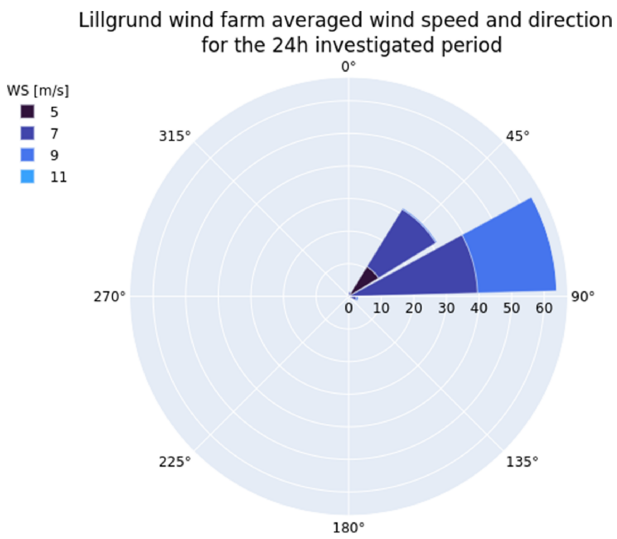
The sDMD algorithm is applied to measurement data from the Lillgrund offshore wind farm. The Supervisory Control and Data Acquisition (SCADA) systems connected to each turbine are the most available and representative of the local flow and operational characteristics, especially for large offshore wind farms. Here, in this study, the active power output signals are extracted from SCADA of every turbine in Lillgrund offshore wind farm at a sampling frequency of 0.5 Hz. The case study aims to fully leverage the availability and the higher frequency of information fed into the online DMD workflow.

Lillgrund consists of 48 2.3MW turbines (see Figure 1A) and is located in Øresund in East Denmark. The perpendicular spacing between the turbines is 4.3*D* and 3.3*D* (where *D* is the rotor diameter of the Siemens SWT-2.3-93 wind turbine), for southwesterly and southeasterly incoming wind sectors, respectively.

A 24-h time series segment, consisting of the power output channels of all 48 turbines, is used as an input for the sDMD algorithm. The daily distribution of the wind speed and direction is presented in Figure 1B. The investigated 24-h period exhibits nonstationary behavior, consisting of strong atmospheric variability compared to typical conditions. It contains a number of wind speed and wind direction gust events, and the overall wind direction rotates by over 40° over 24 h. Turbine wakes generally originate from the northeasterly direction, and sub-rated wind speeds cover the entire region II of the power curve.[44,45] These turbulent conditions enable the performance of the proposed sDMD algorithm to be



(A) Lillgrund offshore wind farm layout and turbine IDs, where rotor diameter D=93m.



(B) Distribution of the investigated 24-hour wind speed and direction in Lillgrund. Atmospheric parameters are spatially averaged over the wind farm. x-axis indicate the percentage of occurrence [%] of the corresponding wind speed and direction bins.

FIGURE 1 Lillgrund wind farm layout and daily distribution of wind speed and power for the investigated period within the case study [Colour figure can be viewed at wileyonlinelibrary.com]

evaluated under prominent aerodynamic interaction between turbines while using the wake effects as a means to propagate information. It also increases the confidence in the resulting forecasts to be used for control applications, particularly for flow control.

The rank of the model reduction is constrained between $r_{min} = 10$ and $r_{max} = 30$, with a memory half-life of $T_{1/2} = 5$ h and a projection error threshold of $\epsilon = 0.28$. Additionally, we investigate the inclusion of delay states and additional input channels (wind speed, wind direction, and turbine rotor speed), as well as performing a parametric study on the choice of $T_{1/2}$.

3.1 | sDMD mode analysis

The sDMD algorithm with a 3-min forecast horizon is applied to the data set. In this section, we analyze the evolution of the sDMD modes. In particular, three snapshots in the time series indicated as Snapshots 1–3 in Figure 2 are further investigated. The events which trigger an update of the orthogonal bases are indicated in Figure 2 by the black vertical lines. It can be observed in the lower plot that the bases tend to update in the vicinity of sudden wind speed or wind direction changes.

The discrete-time eigenvalues at Snapshots 1–3 are plotted on the complex plane in Figure 3. A number of observations can be made. Firstly, the value and quantity of the DMD modes change constantly due to the streaming nature of the DMD algorithm. In this case, there are 14, 23, and 28 eigenvalues at Snapshots 1–3, respectively. Secondly, while the eigenvalues are mostly scattered for each snapshot, there is a single eigenvalue with a value of 1 on the complex plane that remains constant for all snapshots. This particular eigenmode corresponds to the mean value of the system and is the dominant eigenmode. Lastly, there are a number of lightly damped eigenvalues surrounding the real axis with a small complex argument, indicating that these eigenvalues are low frequency. This demonstrates that the algorithm successfully captures low-rank behavior in the data. The eigenvalues with the largest real component correspond to the leading eigenvectors used in the rank reduction step (Table 1), resulting in the updated model containing the most prominent dynamics. It can also be noted that the majority of the discrete eigenmodes lie within the unit circle, indicating a decaying DMD mode. Therefore, if a forecast is performed recursively using the same state transition matrix, the forecast will converge to the value of the constant DMD mode as all other modes decay.

The three leading DMD eigenmodes are visualized over the wind farm layout in Figure 4. The dominant DMD mode (corresponding to a value of 1) clearly highlights the power output variation throughout the wind farm as a result of the average wake effects. The evolution of the wind direction over the time span from approximately east to northeast can be seen visually from the direction of the gradients in Snapshots 1 and 3. From DMD Modes 2 and 3, we observe that the turbine signals are grouped into coherent turbine rows and clusters. This provides insight into the most correlated turbine signals at a given point in time. It should be noted that Figure 4 shows only the real component of the DMD modes.

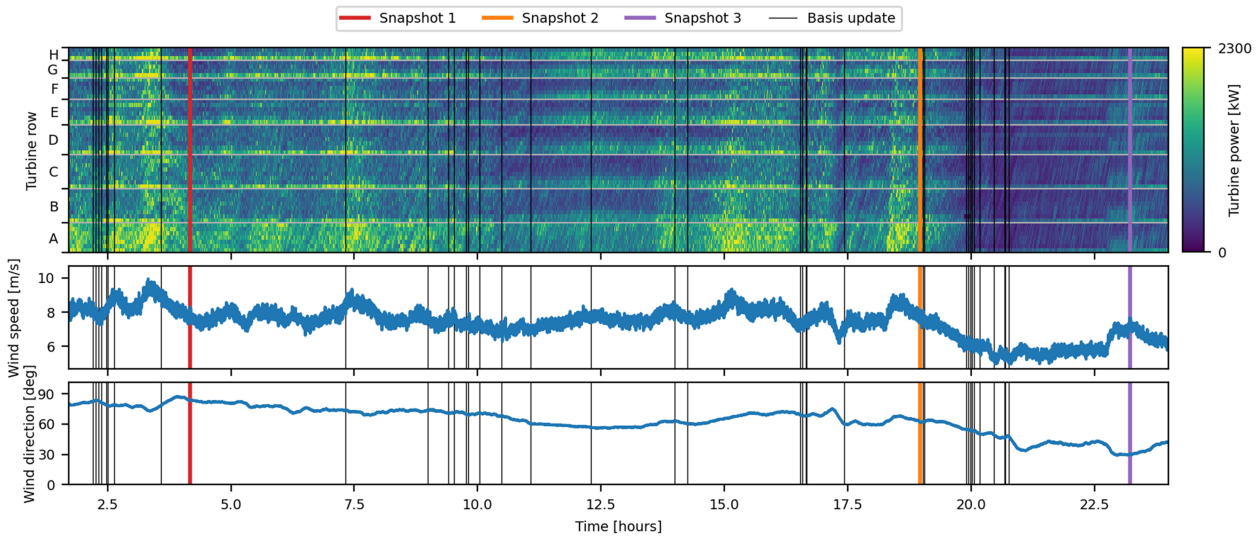


FIGURE 2 Time series of the power output per turbine *(top)* for incoming wind speed and direction *(bottom)* during the investigated 24-h period in Lillgrund offshore wind farm case study. Black vertical lines indicate the update of the orthogonal bases. Turbine column number increases vertically [Colour figure can be viewed at wileyonlinelibrary.com]

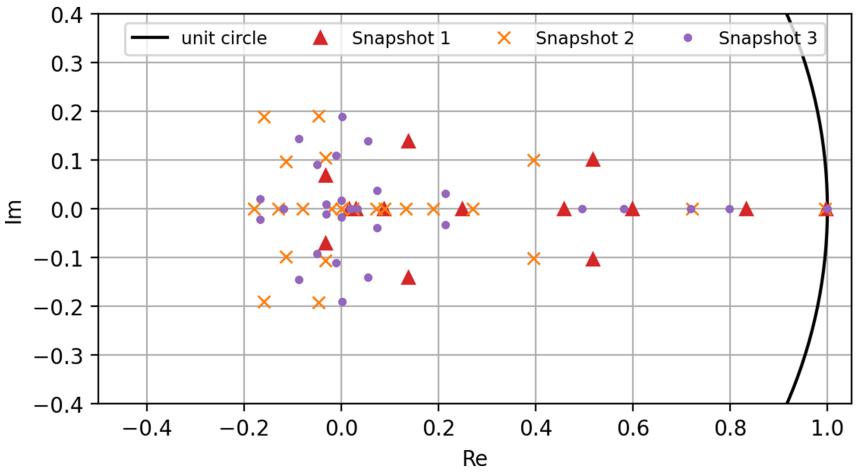


FIGURE 3 DMD eigenvalues plotted on the complex plane using 3-min-ahead sDMD forecasting ($T_{1/2} = 1.7$ h, $r_{min} = 10$, and $r_{max} = 30$). Eigenvalues are plotted for the time steps at the end of Snapshots 1–3, containing 14, 23, and 28 eigenvalues, respectively [Colour figure can be viewed at wileyonlinelibrary.com]

The DMD modes are typically complex, where the imaginary component is able to capture the lagged dynamics of the system, such as wake effects.

3.2 | Forecasting performance

To measure the performance of sDMD forecasting, three reference streaming forecasting methods are used: persistence, moving average, and recursive least squares (RLS). Each of these methods has streaming implementations as summarized in Table 2, making them suitable as a comparison for sDMD. Persistence forecasting is commonly used as a benchmark for forecasting methods due to the high autocorrelation in turbulent flows. For longer wind farm forecasts, it has been suggested to incorporate to the signal mean.[46] For this reason, a moving average filter is also included as a benchmark case. The window length for the moving average filter was chosen as the optimal window for the data set described in Section 3, which was found to be $w = 420$ s. Finally, we include an implementation of the well-known recursive least squares (RLS) adaptive filter due to its similarity with sDMD. As pointed out by Zhang et al., the sDMD is largely equivalent to the RLS filter with a forgetting factor;[33] however, there are two key differences in their implementations. Firstly, sDMD updates a DMD matrix in real time, whereas RLS typically updates a

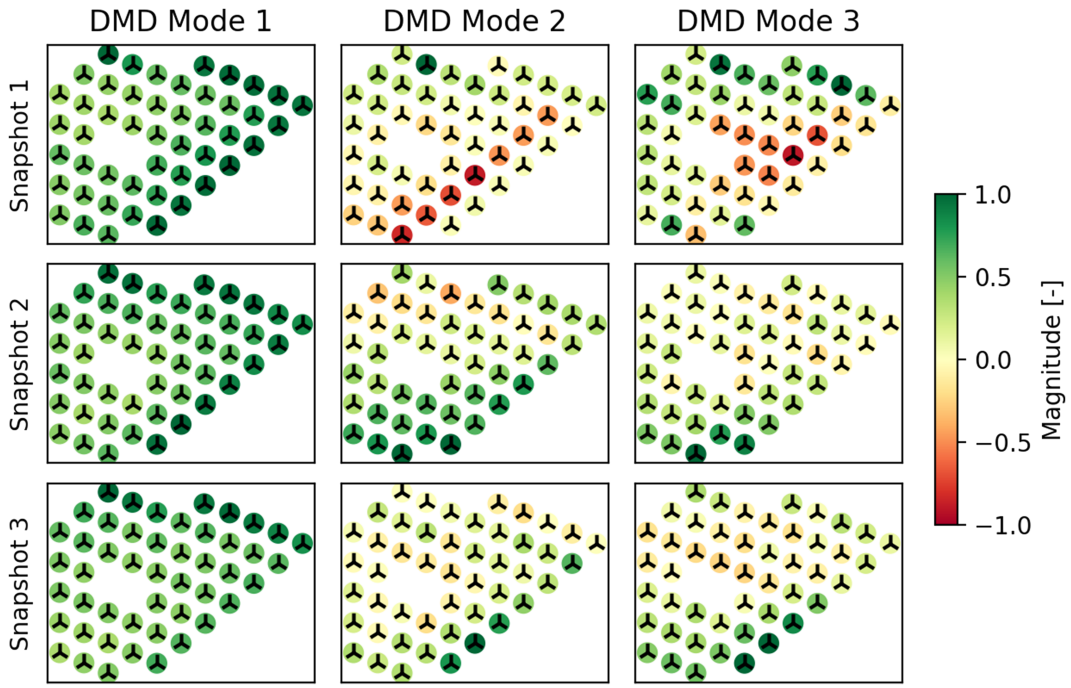


FIGURE 4 Leading DMD eigenmodes at Snapshots 1–3 using 3-min-ahead sDMD forecasting ($T_{1/2} = 1.7$ h, $r_{min} = 10$, and $r_{max} = 30$) [Colour figure can be viewed at wileyonlinelibrary.com]

TABLE 2 Summary of forecasting methods used as a comparison to sDMD forecasting

Persistence	Moving average	Recursive least squares
$\mathbf{x}_{k+f} = \mathbf{x}_k$	$\mathbf{x}_{k+f} = \frac{1}{w}\sum_{i=0}^{w}\mathbf{x}_{k-i}$	$\mathbf{x}_{k+f} = \mathbf{H}_k\mathbf{x}_k$
	where w is the moving window size	where rows of $\mathbf{H}_k \in \mathbb{R}^{n\times n}$ are RLS filter coefficients[27]

coefficient vector. Secondly, the presented sDMD method includes an updating SVD step which acts as a model reduction. By extending RLS for multiple outputs, the RLS method can be considered as sDMD without the model reduction step.

An instance of sDMD forecasting and the three reference forecasting methods is shown in Figure 5, where the power output of Turbine Row A in the Lillgrund wind farm is plotted for 50-min period. Qualitatively, the DMD forecast appears as a low-pass filter on the various signals; however, as identified in the previous section, all signals are coupled via the various DMD eigenmodes. This allows the individual turbine forecasts to act on information from all other signals if an adequate correlation exists. This is in contrast to the persistence and moving average filters, which act on each signal independently. Additionally, compared to persistence and the moving average forecasts, sDMD and RLS have a noticeable improvement in phase response.

To quantify the performance of the forecasts, the root-mean-square error (RMSE) for each turbine forecast relative to the true value is calculated. To further quantify the improvement of the sDMD forecasting, the mean percentage improvement compared to persistence forecasting is calculated. That is, the forecasting performance of method X is

$$\text{performance}_X = \frac{\text{RMSE}_{\text{pers}} - \text{RMSE}_X}{\text{RMSE}_{\text{pers}}} \tag{30}$$

sDMD is able to outperform persistence and moving average forecasting for all turbines in the wind farm as shown in Figure 6. We also observe that the performance varies for each turbine. For a closer insight, the RMSE is plotted over a range of forecasting horizons in Figure 7. Here, we see a marked difference in RMSE between sDMD, persistence forecasting, and a moving average filter, and a slight improvement over the RLS adaptive filter. Persistence forecasting and RLS were found to outperform sDMD for very short time horizons up to 30 s. This is due to the projection error introduced in the sDMD algorithm when transforming from high- to low-dimensional space. Nevertheless, sDMD produces superior forecasts for time horizons above 30 s.

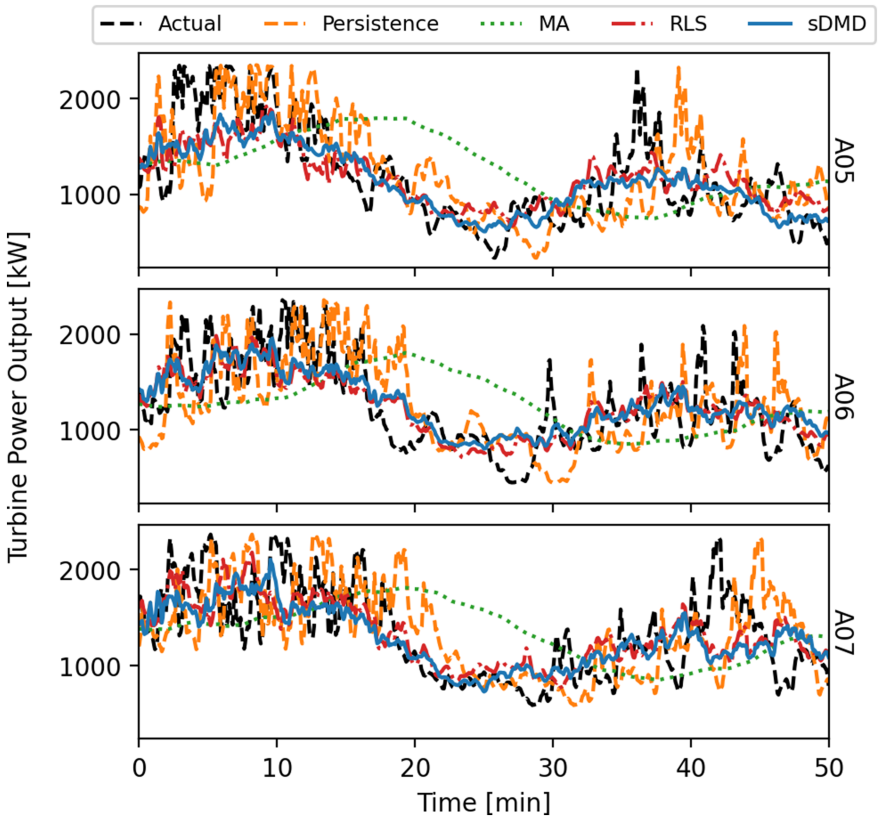


FIGURE 5 Example time series of 3-min-ahead forecasts for the three most downstream turbines in Row A. Forecasting methods shown are persistence, moving average (MA, $w = 420$ s), recursive least squares (RLS, $T_{1/2} = 5$ h), and sDMD ($T_{1/2} = 5$ h, $r_{min} = 10$, and $r_{max} = 30$) [Colour figure can be viewed at wileyonlinelibrary.com]

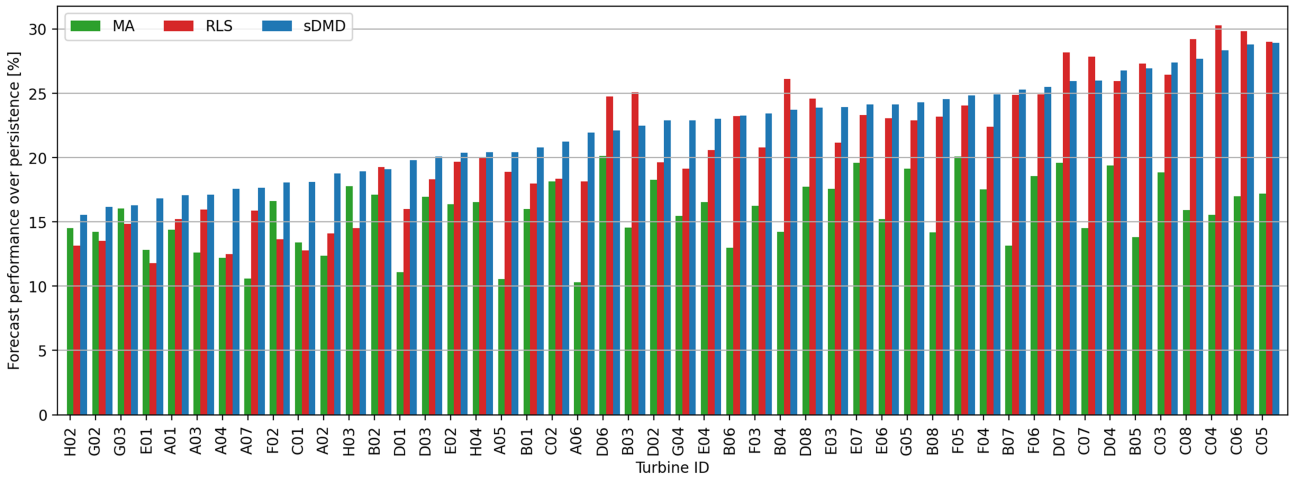


FIGURE 6 Performance of individual turbine forecasts relative to persistence. Forecasting methods shown are moving average (MA, $w = 420$ s), recursive least squares (RLS, $T_{1/2} = 5$ h), and sDMD ($T_{1/2} = 5$ h, $r_{min} = 10$, and $r_{max} = 30$) [Colour figure can be viewed at wileyonlinelibrary.com]

The performance of sDMD and RLS forecasting is particularly sensitive to the memory half-life, $T_{1/2}$, where there is a trade-off between the rate at which new information is adopted and past experience is forgotten. To show this, the average forecast performance over all turbines is shown in Figure 8 as a function of f and $T_{1/2}$ for both RLS and sDMD. We observe that the forecasts outperform persistence for a large range of forecast horizons, particularly in the range of 2 to 10 min. For sDMD, $T_{1/2}$ appears to have an optimal value for this particular data set of approximately 5 h, achieving an improvement of 24% over persistence forecasting. RLS, on the other hand, typically requires a longer memory half-life compared to sDMD to achieve the same level of performance. In this case, we find an optimal value of $T_{1/2} = 5$ h with a performance of 23.9%.

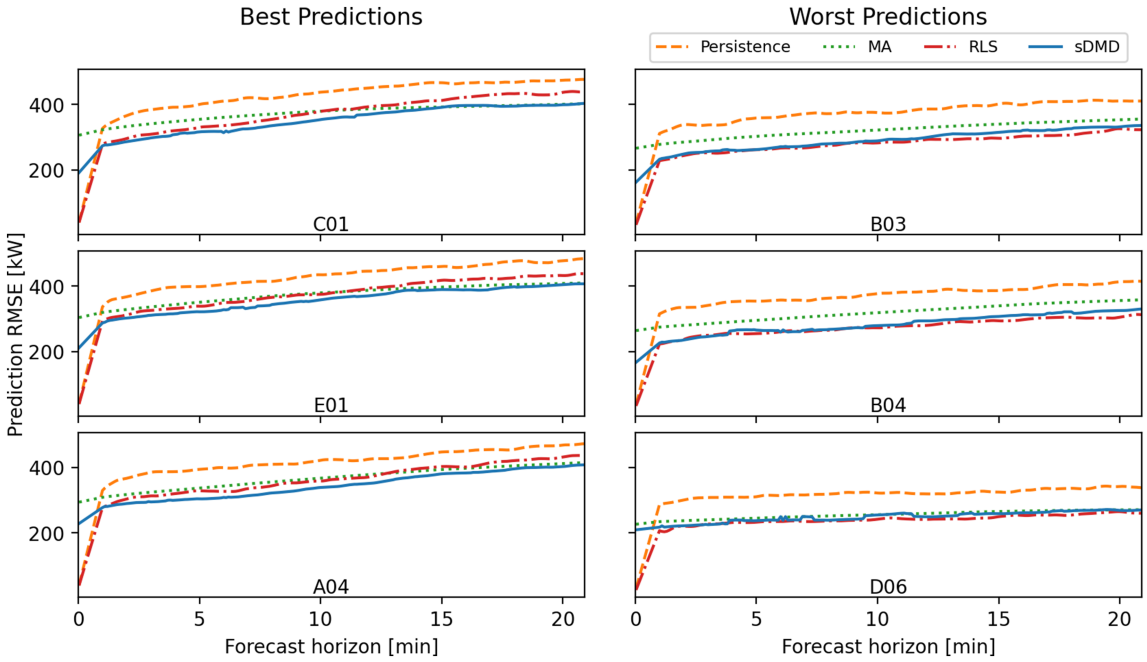
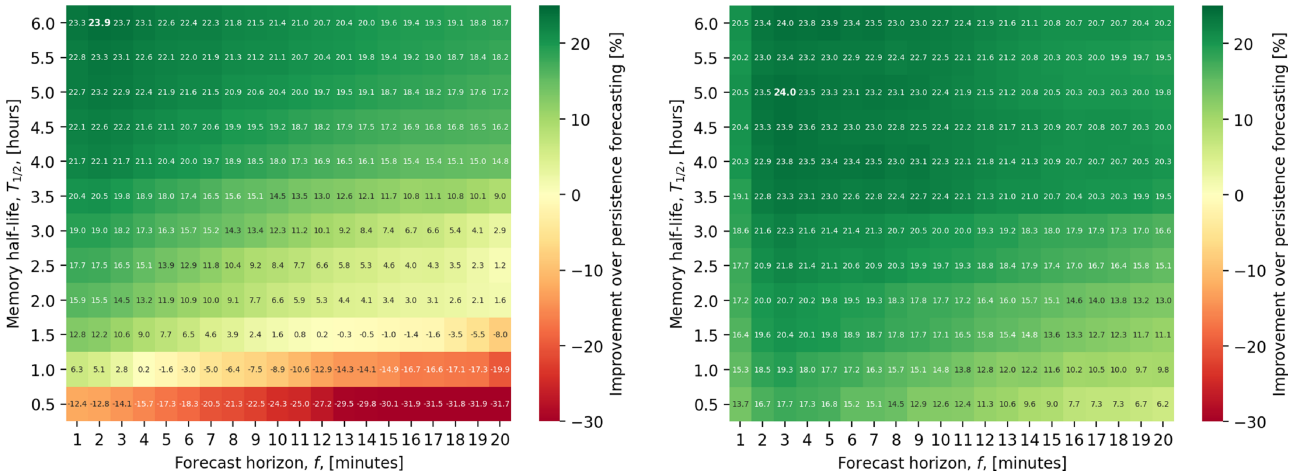


FIGURE 7 RMSE of turbine power output forecast versus forecast horizon for selected wind turbines. Forecasting methods used are persistence, moving average (MA, $w = 420$ s), recursive least squares (RLS, $T_{1/2} = 5$ h), and sDMD ($T_{1/2} = 5$ h, $r_{min} = 10$, and $r_{max} = 30$) [Colour figure can be viewed at wileyonlinelibrary.com]



(A) RLS. **(B)** sDMD.

FIGURE 8 RMSE of forecasts for varying forecast horizons and memory half-lives for (A) RLS and (B) sDMD ($r_{min} = 10$ and $r_{max} = 30$) methods. RMSE is averaged over all turbine forecasts [Colour figure can be viewed at wileyonlinelibrary.com]

The magnitude of $T_{1/2}$ reflects the physical timescale at which the atmospheric dynamics evolve and likely vary based on the application of the algorithm. In addition, Figure 8 shows that a poor choice of tuning parameters could potentially cause the RLS to perform worse than the persistence method, while the sDMD shows better consistency in its performance despite the different choices of memory half-life and forecast horizon.

3.3 | State augmentation

We finally compare the percentage improvement of RMSE over persistent forecasting for variations of sDMD (Table 3). Namely, the inclusion of additional channels and delay states as described in Section 2.4. The additional channels used in this study are Nacelle wind speed, Nacelle

TABLE 3 Average forecasting performance (relative to persistence forecasting) of various methods, including moving average (MA, $w = 420$), recursive least squares (RLS, $T_{1/2} = 5$ h), and sDMD ($T_{1/2} = 5$ h, $r_{min} = 10$, and $r_{max} = 30$) with additional channels (AC) and delay states (DS)

Forecast horizon	MA	RLS	sDMD	sDMD with DS	sDMD with AC	sDMD with AC + DS
1 min	11.20%	22.68%	20.53%	20.44%	18.97%	18.96%
3 min	16.70%	22.94%	24.02%	24.01%	23.69%	23.68%
5 min	16.36%	21.90%	23.31%	23.35%	23.48%	23.48%
15 min	15.10%	18.75%	20.52%	20.76%	22.17%	22.16%
20 min	15.08%	17.24%	19.80%	20.14%	21.90%	21.90%

Note: This table uses the performance measure defined in Equation (30).

direction, and turbine rotor speed. Each of the 48 turbines provides these three signals as well as the power output of the turbine, giving the augmented state size of 192 elements. Without state augmentation, sDMD outperforms 5-min persistence forecasting of the power of each turbine on average by 23.31%, which is a marked improvement over moving average filter and RLS, which show a 16.32% and a 21.90% improvement, respectively. An additional yet modest gain can be achieved by including additional channels or a single delay state. By including both, the performance of the sDMD algorithm reaches 23.48% in this investigation. It was found that adding more than one delay state leads to a reduction in performance for the presented problem. Although it is expected that augmenting the input snapshots would yield better forecasts, it was found that sDMD is able to adequately characterize the wind farm system without any augmentations, although this will likely vary depending on atmospheric stability and turbulence levels.

4 | DISCUSSION

The key advantage of the sDMD algorithm over batch-trained forecasters lies in its ability to adjust itself on the fly, identifying new correlations between the data channels as they appear, and forgetting old behavior which is no longer relevant. Additionally, its data-driven approach allows the algorithm to capture complex interactions between turbines in a wind farm without any prior model. There is a trade-off, however, as the algorithm is unable to predict scenarios it has not observed previously, or for a long period of time. For example, if a wind farm initiates a yaw steering sequence for the first time, it is expected that the sDMD algorithm will take some time before it is able to recognize the new behavior and incorporate it into its model.

The sDMD algorithm is able to outperform persistence and moving average power forecasting for at least 20 min into the future, as well as showing a small improvement over RLS forecasting. With a memory half-life of 5 h, sDMD is able to forecast the power output of each turbine in a wind farm with a performance increase (relative to persistence forecasting) of 24.0% for 3-min forecasts and 19.8% for 20-min forecasts. This is in contrast to recursive least squares with a performance increase of 22.9% and 17.2%, respectively. Despite the promising results for several-minute forecasting, sDMD was unable to show improvements compared to persistence and RLS forecasting for timescales of less than 30 s. The reason for this is attributed to the model reduction, which will cause some information to be lost in forecasts of the range of seconds. This is compensated in the minute-range forecasts, where sDMD outperforms all benchmarks as sDMD is able to capture the dominant low-rank behavior of the system. A scenario in which sDMD and RLS do not outperform persistence or moving average forecasting is during periods above rated wind speed and during shutdowns. These scenarios present an almost constant signal, where persistence forecasting cannot be beaten. Nevertheless, the authors found the sDMD algorithm to be robust as it continued to operate, albeit poorly, in these scenarios and quickly adjust to new conditions once the wind farm deviates from rated output or shutdown conditions (Table 3).

Compared to RLS, sDMD requires a shorter memory half-life to achieve the same level of performance in minute-range forecasts, allowing sDMD to adapt its forecasts to changing conditions at a faster rate. Secondly, while RLS is already quite computationally efficient, the time complexity of sDMD for each update is lower than RLS, especially for large state sizes as outlined in Section 2.5. Finally, sDMD tends to be more robust than RLS for rank-deficient data streams. That is, the full-rank condition in Equation (20) is violated if there are redundant signals (row deficiency) or an insufficient amount of snapshots compared to variables (column deficiency), leading to an ill-conditioned system. By performing a model reduction, and with an appropriate choice of r_{min} and r_{max}, sDMD alleviates this issue, whereas RLS may produce numerically unstable forecasts.

The parameters of the sDMD algorithm, in particular, r_{min}, r_{max}, ϵ, and $T_{1/2}$ should be chosen with care for the given problem. A range of values for r_{min} and r_{max} can typically be determined by performing a preliminary DMD on a data subset, where the user can identify the number of dominant DMD modes. The choice of ϵ largely depends on the nature of the data and should be chosen so that basis updates occur when significant changes in the dynamic system occur. This may require a degree of trial and error. From the presented study, the memory half-life, $T_{1/2}$, appears to have an optimal value of approximately 5 h. This is likely dictated by the timescales present in the atmospheric conditions. These

parameters are closely linked with the chosen data set in this investigation. While the data set was chosen for its wide range of wind conditions, results will likely vary for different wind farm layouts and wind conditions.

An application of sDMD, which is not explored in this investigation, is its use in wind farm control. By applying the a state augmentation as described in Section 2.4, sDMD can be applied along with the concept of DMD with control[39] to incrementally estimate the reduced-order state transition matrix, $\tilde{\mathbf{A}}_k$, and the input matrix, $\tilde{\mathbf{B}}_k$, thus forming the state-space system:

$$\tilde{\mathbf{x}}_{k+1} = \tilde{\mathbf{A}}_k \tilde{\mathbf{x}}_k + \tilde{\mathbf{B}}_k \mathbf{u}_k \tag{31}$$

In future investigations, this can be used for control applications, such as model predictive control or Kalman filtering in a wind farm.

5 | CONCLUSIONS

This paper demonstrates a streaming dynamic mode decomposition algorithm (sDMD) for forecasting and system identification of a wind farm system. In particular, we focus on short-term power output forecasts of each turbine in a wind farm. sDMD is applied to a 24-h data set from the Lillgrund wind farm at a 0.5-Hz sampling rate. A key innovation of using sDMD is its ability to adapt to changing wind conditions such as wind speed, wind direction, or turbulence changes. These adaptations occur in real time as new measurements are obtained, allowing the algorithm to operate continuously and without downtime. A clear improvement in forecasting can be seen compared to persistence, moving average, and recursive least squares forecasting. We observe a performance increase (relative to persistence forecasting) of 24.0% for 3-min forecasts and 19.8% for 20-min forecasts. This is in contrast to recursive least squares forecasting with a performance increase of 22.9% and 17.2%, respectively. By including state augmentations, such as additional input channels and delay states, additional forecasting improvements are possible; however, for turbulent wind conditions, these are negligible in comparison to the initial improvement of using sDMD. While the focus of the investigation is on short-term forecasting, the sDMD algorithm additionally provides a linear state-space model of the wind farm system, which has potential applications in wind farm control.

ACKNOWLEDGEMENTS

The work is funded by the WISDOM Project (EUROSTARS 12842). The authors would like to thank Anders Sommer from Vattenfall A/S for the original provision of the Lillgrund SCADA data and Elliot Simon from DTU Wind Energy for initial post-processing and management of the database that is collected under the TotalControl EU Horizon 2020 project.

CONFLICT OF INTERESTS

The authors declare that they have no conflicts of interest.

AUTHOR CONTRIBUTIONS

J. L. developed and implemented the sDMD algorithm and performed the analysis of results. T. G. defined and processed the case study data used to best demonstrate the sDMD algorithm. All the authors contributed to the conceptualization, investigation, and reporting of the research presented in this paper.

PEER REVIEW

The peer review history for this article is available at https://publons.com/publon/10.1002/we.2694.

DATA AVAILABILITY STATEMENT

A Python implementation of the sDMD algorithm is available at https://doi.org/10.5281/zenodo.4646749.[43]

Lillgrund data subject to third party restrictions.

ORCID

Jaime Liew https://orcid.org/0000-0002-5858-4614
Tuhfe Göçmen https://orcid.org/0000-0002-2510-0388
Wai Hou Lio https://orcid.org/0000-0002-3946-8431

REFERENCES

1. Veers P, Dykes K, Lantz E, et al. Grand challenges in the science of wind energy. *Science*. 2019;366(6464):eaau2027.
2. Bossanyi EA. Short-term wind prediction using Kalman filters. *Wind Eng*. 1985;9(1):1-8. http://www.jstor.org/stable/43749025

3. Matevosyan J, Soder L. Minimization of imbalance cost trading wind power on the short-term power market. *IEEE Trans Power Syst*. 2006;21(3):1396-1404.
4. Archer CL, Vasel-Be-Hagh A, Yan C, et al. Review and evaluation of wake loss models for wind energy applications. *Appl Energy*. 2018;226:1187-1207.
5. Madsen HA, Larsen TJ, Pirrung GR, Li A, Zahle F. Implementation of the blade element momentum model on a polar grid and its aeroelastic load impact. *Wind Energy Sci*. 2020;5(1):1-27.
6. Jensen NO. A note on wind generator interaction; 1983.
7. Frandsen S, Barthelmie R, Pryor S, et al. Analytical modelling of wind speed deficit in large offshore wind farms. *Wind Energy: An Int J Progress Appl Wind Power Conv Technol*. 2006;9(1-2):39-53.
8. Bastankhah M, Porté-Agel F. A new analytical model for wind-turbine wakes. *Renew Energy*. 2014;70:116-123.
9. Larsen GC, Madsen HA, Thomsen K, Larsen TJ. Wake meandering: a pragmatic approach. *Wind Energy: An Int J Progress Appl Wind Power Conv Technol*. 2008;11(4):377-395.
10. Pinson P. Very-short-term probabilistic forecasting of wind power with generalized logit–normal distributions. *J R Stat Soc Ser C (Appl Stat)*. 2012;61(4):555-576.
11. Dowell J, Pinson P. Very-short-term probabilistic wind power forecasts by sparse vector autoregression. *IEEE Trans Smart Grid*. 2016;7(2):763-770.
12. Voyant C, Notton G, Kalogirou S, et al. Machine learning methods for solar radiation forecasting: a review. *Renew Energy*. 2017;105:569-582.
13. Yang L, He M, Zhang J, Vittal V. Support-vector-machine-enhanced Markov model for short-term wind power forecast. *IEEE Trans Sustain Energy*. 2015;6(3):791-799.
14. Deo RC, Wen X, Qi F. A wavelet-coupled support vector machine model for forecasting global incident solar radiation using limited meteorological dataset. *Appl Energy*. 2016;168:568-593.
15. Persson C, Bacher P, Shiga T, Madsen H. Multi-site solar power forecasting using gradient boosted regression trees. *Solar Energy*. 2017;150:423-436.
16. Huang J, Troccoli A, Coppin P. An analytical comparison of four approaches to modelling the daily variability of solar irradiance using meteorological records. *Renew energy*. 2014;72:195-202.
17. Lahouar A, Slama JBH. Hour-ahead wind power forecast based on random forests. *Renewable energy*. 2017;109:529-541.
18. Wang H, Lei Z, Zhang X, Zhou B, Peng J. A review of deep learning for renewable energy forecasting. *Energy Conv Manag*. 2019;198:111799. https://linkinghub.elsevier.com/retrieve/pii/S0196890419307812
19. Göçmen T, Meseguer Urbán A, Liew J, Lio AWH. Model-free estimation of available power using deep learning. *Wind Energy Sci*. 2021;6(1):111-129. https://wes.copernicus.org/articles/6/111/2021/
20. Ben Bouallègue Z, Heppelmann T, Theis SE, Pinson P. Generation of scenarios from calibrated ensemble forecasts with a dual-ensemble copula-coupling approach. *Monthly Weather Rev*. 2016;144(12):4737-4750.
21. Eide SS, Bremnes JBR, Steinsland I. Bayesian model averaging for wind speed ensemble forecasts using wind speed and direction. *Weather Forecast*. 2017;32(6):2217-2227.
22. Andersson LE, Imsland L. Real-time optimization of wind farms using modifier adaptation and machine learning. *Wind Energy Sci*. 2020;5(3):885-896. https://wes.copernicus.org/articles/5/885/2020/
23. Göçmen T, Giebel G. Data-driven wake modelling for reduced uncertainties in short-term possible power estimation. *J Phys Conf Ser*. 2018;1037:72002. https://doi.org/10.1088/1742-6596/1037/7/072002
24. Göçmen T, Giebel G, Poulsen NK, Sørensen PE. Possible power of down-regulated offshore wind power plants: the PossPOW algorithm. *Wind Energy*. 2019;22(2):205-218. https://onlinelibrary.wiley.com/doi/abs/10.1002/we.2279
25. Haykin SS. *Adaptive Filter Theory*: Pearson Education India; 2008.
26. Björck AA. *Numerical Methods for Least Squares Problems*: SIAM; 1996.
27. Staub R, Steinmann SN. Efficient recursive least squares solver for rank-deficient matrices. *Appl Math Comput*. 2021;399:125996.
28. Schmid PJ. Dynamic mode decomposition of numerical and experimental data. *J Fluid Mech*. 2010;656:5-28.
29. Tu JH, Rowley CW, Luchtenburg DM, Brunton SL, Kutz JN. On dynamic mode decomposition: theory and applications. arXiv preprint arXiv:13120041; 2013.
30. Annoni J, Taylor T, Bay C, et al. Sparse-sensor placement for wind farm control. *J Phys Conf Ser*. 2018;1037(3):32019.
31. Fortes-Plaza A, Campagnolo F, Wang J, Wang C, Bottasso CL. A POD reduced-order model for wake steering control. *J Phys: Conf Ser*. 2018;1037(3):32014.
32. Cassamo N, van Wingerden J-W. On the potential of reduced order models for wind farm control: a Koopman dynamic mode decomposition approach. *Energies*. 2020;13(24):6513.
33. Zhang H, Rowley CW, Deem EA, Cattafesta LN. Online dynamic mode decomposition for time-varying systems. *SIAM J Appl Dyn Syst*. 2019;18(3):1586-1609.
34. Matsumoto D, Indinger T. On-the-fly algorithm for dynamic mode decomposition using incremental singular value decomposition and total least squares. arXiv preprint arXiv:170311004; 2017.
35. Hemati MS, Williams MO, Rowley CW. Dynamic mode decomposition for large and streaming datasets. *Phys Fluids*. 2014;26(11):111701.
36. Vasconcelos Filho E, dos Santos PL. A dynamic mode decomposition approach with Hankel blocks to forecast multi-channel temporal series. *IEEE Control Syst Lett*. 2019;3(3):739-744.
37. Tu JH, Rowley CW, Luchtenburg DM, Brunton SL, Kutz JN. On dynamic mode decomposition: theory and applications. *J Comput Dyn*. 2014;1(2):391-421.
38. Arbabi H, Mezic I. Ergodic theory, dynamic mode decomposition, and computation of spectral properties of the Koopman operator. *SIAM J Appl Dyn Syst*. 2017;16(4):2096-2126.
39. Proctor JL, Brunton SL, Kutz JN. Dynamic mode decomposition with control. *SIAM J Appl Dyn Syst*. 2016;15(1):142-161.
40. Penrose R. A generalized inverse for matrices. *Mathematical Proceedings of the Cambridge Philosophical Society*, Vol. 51: Cambridge University Press; 1955:406-413.
41. Kutz JN, Brunton SL, Brunton BW, Proctor JL. *Dynamic Mode Decomposition: Data-Driven Modeling of Complex Systems*: SIAM; 2016.

42. Povey D, Cheng G, Wang Y, et al. Semi-orthogonal low-rank matrix factorization for deep neural networks. *Proc. Interspeech*. 2018:3743-3747. https://doi.org/10.21437/Interspeech.2018-1417
43. Liew J. *Streaming-DMD*: Zenodo; 2021. https://doi.org/10.5281/zenodo.4646749
44. Göçmen T, Giebel G. Estimation of turbulence intensity using rotor effective wind speed in Lillgrund and Horns Rev-I offshore wind farms. *Renew Energy*. 2016;99:524-532. https://www.sciencedirect.com/science/article/pii/S0960148116306346
45. Pedersen MM, Larsen GC. Integrated wind farm layout and control optimization. *Wind Energy Sci*. 2020;5(4):1551-1566. https://wes.copernicus.org/articles/5/1551/2020/
46. Nielsen TS, Joensen A, Madsen H, Landberg L, Giebel G. A new reference for wind power forecasting. *Wind Energy: An Int J Progress Appl Wind Power Conv Technol*. 1998;1(1):29-34.

How to cite this article: Liew J, Göçmen T, Lio WH, Larsen GC. Streaming dynamic mode decomposition for short-term forecasting in wind farms. *Wind Energy*. 2022;25(4):719-734. doi:10.1002/we.2694